

Защитная речь — Smart Price (v4, ~10 мин)

Простыми словами, без слайд-нарратива.

Не озвучиваем то, что и так нарисовано на экране — говорим о смысле и результатах.

[в курсиве] — можно опустить если поджимает время.

Слайд 1 · Титульный (0:00 — 0:30)

Здравствуйте, уважаемая комиссия.

Меня зовут Санько Егор, группа 4-МД-16. Тема моей выпускной работы — **система метапоиска товаров на торговых площадках с интеллектуальным анализом цен**.
Научный руководитель — Вагнер Виктория Игоревна.

Слайд 2 · Актуальность · Цель и задачи (0:30 — 2:00)

Когда покупателю нужно найти лучшую цену в России или Беларуси, он открывает пять-шесть вкладок и сравнивает руками. На это уходит **от восьми до пятнадцати минут**.

Дополнительная проблема — цены на маркетплейсах меняются по несколько раз в день: разница на один и тот же товар легко достигает **двадцати-тридцати процентов**. Общего сервиса, который покрывал бы и российские, и белорусские площадки в одном окне, не существует.

Цель работы — спроектировать и сделать такой сервис.

Я разбил её на пять задач: проанализировать рынок и существующие решения, спроектировать архитектуру, написать парсеры для шести маркетплейсов, встроить искусственный интеллект для анализа цен, и развернуть систему в продакшен.

Слайд 3 · Решение (2:00 — 2:40)

Главная идея решения простая: **опрашивать шесть маркетплейсов одновременно и показывать результаты по мере их прихода**, не дожидаясь самого медленного.

Параллельно работает искусственный интеллект — он смотрит на найденную цену, сравнивает с историей за последние два месяца и подсказывает, дёшево это сейчас или дорого.

Слайд 4 · Поиск (2:40 — 3:10)

Это рабочий пример. Пользователь ввёл просторечное «плойка пять» — ИИ привёл это к нормальной форме «PlayStation 5», система запустила поиск и нашла **тридцать восемь предложений из пяти источников**. Лучшая цена на Яндекс.Маркете — тридцать девять тысяч рублей.

Слайд 5 · Архитектура (3:10 — 4:10)

Система состоит из пяти слоёв.

Браузер пользователя — это **Next.js** на TypeScript. Он подписывается на поток событий от сервера и дорисовывает карточки по одной по мере прихода.

Перед сервером стоит **nginx** с включённым HTTP/2. Важный момент: по умолчанию nginx копит ответы и отдаёт их одним куском — это убивает потоковую передачу. Пришлось специально его настроить, чтобы ответы шли мгновенно.

Сам сервер — **FastAPI на Python**. Он запускает шесть парсеров параллельно одной командой и сразу отправляет каждый найденный товар клиенту, не дожидаясь остальных.

[Для базы данных используется PostgreSQL, для очередей фоновых задач — Redis с Celery. Всё развёрнуто как шесть Docker-контейнеров на одном сервере.]

Слайд 6 · Парсеры (4:10 — 4:45)

Из **двадцати пяти изученных маркетплейсов** я отобрал шесть — по качеству данных и стабильности работы.

Onliner — белорусский, у него удобный открытый API. Яндекс.Маркет — основной российский источник. Ретард и World Devices — крупные российские магазины электроники. И две площадки, которые специализируются на технике Apple — 1click и BigGeek.

Каждый парсер — отдельный модуль с одинаковым интерфейсом. Чтобы добавить новый магазин, нужно написать только этот один модуль, ядро системы не трогается.

Слайд 7 · Искусственный интеллект (4:45 — 5:20)

В качестве модели я выбрал **Gemini 2.5 Flash от Google**, доступ через сервис OpenRouter. Причины три: хорошо понимает русский, поддерживает потоковую генерацию ответов и стоит **в десять-пятнадцать раз дешевле** GPT-4.

Модель работает по принципу «вызова инструментов» — она сама решает, что нужно ей сейчас: посмотреть историю цен, сравнить предложения или посчитать медиану по рынку. Это значит, что она не выдумывает цифры — все числа берутся из реальной базы.

Слайд 8 · «Найти дешевле» — реверс Алисы (5:20 — 6:20)

Это **уникальная функция, аналогов на рынке нет**.

Идея простая: голосовой ассистент Яндекса умеет искать «то же самое дешевле». У него есть внутренний канал, по которому он получает предложения от мелких магазинов. Публичного интерфейса к этому каналу не существует — но я разобрал, как Алиса с ним общается, и научил свой воркер делать то же самое.

Технически — это **обратная инженерия двоичного протокола поверх WebSocket**. Воркер устанавливает соединение под видом обычного клиента Яндекса, получает в ответ двоичный поток, разбирает его и достаёт из него предложения.

Конкретный пример на скриншоте: айфон семнадцать про найден за **сто девятнадцать тысяч**, что на **шестнадцать тысяч дешевле**, чем на Озоне — это минус двенадцать процентов на одной покупке. Фича работает в продакшене.

[Здесь должна быть гордость в голосе — это центральная часть работы.]

Слайд 9 · Что работает за кулисами (6:20 — 7:00)

Помимо основного поиска, в системе четыре скрытых от глаз подсистемы.

Первая — **исправление запросов**. Пользователь пишет «айфон шестнадцать пра» — модель приводит к «iPhone 16 Pro». Точность девяносто семь из ста.

Вторая — **отсев лишнего**. Если ищешь «iPhone», в выдачу не должны попадать чехлы и зарядки. Я сделал двухступенчатый фильтр: сначала простые правила, потом ИИ-классификация. Качество выдачи выросло на тридцать-сорок процентов.

Третья — **AI-ассистент** для развёрнутого диалога о товарах. Чтобы он не выдумывал цифры, я ограничил его реальными функциями бэкенда. Из десяти провокационных тестов он не соврал ни в одном.

Четвёртая — **история цен** за шестьдесят дней. Это основа, на которой ИИ говорит «дорого» или «дёшево».

Слайд 10 · Сравнение и анализ цены (7:00 — 7:25)

Кроме поиска работают ещё две функции. **Сравнение** — выбираешь до четырёх товаров, ИИ объясняет в чём разница. И **анализ цены** — даёт оценку конкретного предложения по стобалльной шкале относительно медианы рынка с автоматическим определением региона.

Слайд 11 · Цифры (7:25 — 7:50)

Здесь самое важное — **все цифры замерены на боевом сервере**, а не теоретические.

Первая карточка появляется через секунду и четыре десятых. Шесть источников опрашиваются одновременно. Поиск стал быстрее ручного примерно в сто раз. Объём кода — около пятнадцати тысяч строк.

Слайд 12 · Экономика (7:50 — 8:30)

Сервис в эксплуатации **дешёвый**. Сервер — пятнадцать долларов в месяц. API искусственного интеллекта — ещё пять. Домен и сертификат — два. **Итого двадцать два доллара в месяц** за полностью рабочую систему.

Что это даёт пользователю: одна сессия поиска экономит ему десять-тридцать минут времени и от пяти до сорока процентов на цене товара. **Трёх-девяти таких поисков достаточно, чтобы окупить весь месяц работы сервиса.**

[Возможные пути монетизации — реклама от маркетплейсов, партнёрские отчисления или платная подписка с расширенной аналитикой.]

Слайд 13 · Что было сложно (8:30 — 9:25)

Четыре основных инженерных задачи, ради которых работа и существовала.

Первое — обход защит маркетплейсов. У каждой площадки своя защита от ботов, универсального решения нет. Для каждого магазина пришлось придумать свой подход: где-то открытый API, где-то автоматический браузер, где-то ротация заголовков. В итоге **шесть парсеров стабильно работают на проде.**

Второе — потоковая передача результатов. Это та самая настройка nginx, о которой я говорил. Если её не сделать, пользователь увидит всё одним куском в конце. С правильной настройкой первая карточка приходит через секунду и четыре десятых.

Третье — реверс Алисы, о чём уже было.

Четвёртое — самая свежая правка, перед самой защитой. Раньше поиск «чехол iPhone 16» возвращал четыре результата вместо двадцати пяти — система слишком агрессивно отсеивала аксессуары, даже когда пользователь явно их искал. Я добавил **определитель смысла запроса**: семьдесят пять правил, которые до обращения к ИИ понимают, что человек хочет именно чехол, а не телефон. Результат — стало двадцать пять штук, **рост в шесть раз.**

[Если поджимает — четвёртый пункт можно опустить.]

Слайд 14 · Заключение (9:25 — 9:55)

Подведу итог.

Цель работы достигнута. Платформа спроектирована, реализована и работает в продакшене на адресе **smrt-price.ru**. Прямо сейчас она обрабатывает реальные запросы пользователей.

Все **пять задач** работы выполнены. Все ключевые решения подтверждены измерениями: время отклика секунда четыре, шесть источников, ускорение в сто раз по сравнению с ручным поиском, стоимость эксплуатации двадцать два доллара в месяц.

Слайд 15 · Спасибо (9:55 — 10:05)

Доклад окончен. Благодарю за внимание. Готов ответить на ваши вопросы.

Тайм-чарт

№	Слайд	Время	Длит.	Главная мысль
1	Титул	0:00	0:30	Кто я, тема, руководитель
2	Актуальность · Цель · Задачи	0:30	1:30	8–15 мин ручного → 5 задач
3	Решение	2:00	0:40	6 источников одновременно + AI
4	Поиск	2:40	0:30	«плойка 5 → PlayStation 5»
5	Архитектура	3:10	1:00	5 слоёв, потоковая выдача
6	Парсеры	4:10	0:35	25 → 6, модульно
7	ИИ	4:45	0:35	Gemini, в 10–15× дешевле GPT-4
8	Найти дешевле	5:20	1:00	iPhone 17: –16 113 ₽ vs Ozon
9	За кулисами	6:20	0:40	4 подсистемы
10	Сравнение/Анализ	7:00	0:25	95/100 баллов
11	Цифры	7:25	0:25	1,4 с · ~100×
12	Экономика	7:50	0:40	\$22/мес · окупаемость 3–9
13	Что было сложно	8:30	0:55	4 задачи, intent-фильтр 4 → 25
14	Заключение	9:25	0:30	Цель достигнута
15	Спасибо	9:55	0:10	Вопросы
Итого			~10:05	

Шпаргалка цифр (для вопросов)

Метрика	Значение
Время до первой карточки	1,4 с
Полная выдача	24,2 с
Источников активно	6
Изучено маркетплейсов	25
Ускорение vs ручной	~100x
Стоимость сервиса	~\$22/мес
Экономия пользователю	\$2,5–7,5 за поиск
Окупаемость	3–9 использований
Точность исправления запроса	97 / 100
Рост релевантности фильтрации	+30–40%
Защита от выдумывания цифр	9 / 10 тестов
Экономия через Алису	10–40%
iPhone 17 Pro vs Ozon	–16 113 ₽ (–12%)
Intent-фильтр «чехол iPhone»	4 → 25 результатов
Объём кода	~15 000 строк

Заготовки ответов на вопросы

«Что такое SSE и почему вы его выбрали?»

SSE — это «отправка событий с сервера», стандарт поверх обычного HTTP. Сервер сам шлёт обновления клиенту, не дожидаясь запроса. Я выбрал его, потому что в моей задаче нужна только односторонняя передача — сервер пушит карточки по мере их прихода, обратно от клиента ничего слать не нужно. Альтернатива — WebSocket — была бы избыточна: это двусторонний канал, требует дополнительную библиотеку и хуже работает через прокси. У SSE же есть автореконнект из коробки, и он прозрачно проходит через nginx.

«Реверс Алисы — это законно?»

Это исследовательский разбор протокола в рамках академической работы. Никакие условия использования не нарушены: я работаю с трафиком собственного браузерного клиента, который любой пользователь Яндекса может увидеть у себя в инструментах разработчика.

«А если Яндекс изменит свой протокол?»

Это признаваемый риск. Функция «Найти дешевле» вынесена в отдельный воркер и изолирована от остального. Если протокол поменяется — отключится только она, основной поиск по шести источникам продолжит работать. Также предусмотрен мониторинг, который поймает изменения.

«Почему именно Gemini, а не GPT-4 или Claude?»

Три причины. Gemini Flash дешевле GPT-4 примерно в десять раз и Claude в два с половиной раза при сопоставимом качестве для моих задач. Через OpenRouter решается проблема с доступом из России. И если Gemini перестанет устраивать — я могу переключиться на любую другую модель за минуту, не меняя код.

«Почему нет Wildberries и Ozon?»

Wildberries был временно отключён — у него ненадёжные поля наличия товара и в выдачу попадают восстановленные устройства вместе с новыми. Ozon изначально не подключали из-за агрессивной защиты от ботов, требующей отдельной инфраструктуры. Архитектура такова, что добавить их можно в любой момент без переработки ядра.

«Что такое intent-фильтр и почему вы его делали?»

Это последняя правка перед защитой. Раньше система не понимала разницу между запросом «iPhone» и «чехол на iPhone» — оба запроса для парсеров означали «отсеять всё, что выглядит как аксессуар». Я добавил **определитель смысла запроса**: семьдесят пять простых правил, которые ещё до обращения к ИИ понимают, что человек хочет именно чехол. Это даёт стопроцентно повторяемое поведение и закрыло реальный продакшен-баг.

«Какая нагрузка на сервер?»

На пиках до пятидесяти процентов загрузки процессора при пяти-десяти одновременных поисках. Бутылочное горлышко — не процессор, а сами маркетплейсы, их ответ занимает от секунды до двух. По памяти — занято примерно полтора гигабайта из четырёх доступных.

«Будет ли мобильное приложение?»

Архитектура изначально это поддерживает — фронтенд общается с сервером по обычному API. Мобильное приложение на Expo / React Native — это отдельная итерация, она подключается к тому же серверу без изменений на стороне бэкенда.

Финальные напоминания

- **Главные акценты по голосу:** слайд 2 («8–15 минут»), слайд 8 («–16 113 ₽ дешевле, чем на Ozon»), слайд 11 («измерено в проде»), слайд 13 («рост в шесть раз»).
- **Если поджимает время** — опускай *[курсивные блоки]*, особенно про intent-фильтр и базу данных. Это даёт ~25 секунд запаса.
- **Не торопись.** 10 минут — это нормально для защиты. Лучше пауза, чем скороговорка.
- **На последнем слайде** — короткая пауза после «вопросы», кивок, садись.